REQ 5: Stateful creatures

HaozeHou: Framework & Camouflaged State

Design 1

Implement all state logic and transition conditions directly within the `SporeBeast` class using conditional statements (e.g., if-else or switch cases) and integer flags.

Pros	Cons
Simpler initial setup, as all logic is contained within a single file, making it easy to trace the basic execution flow without navigating multiple classes.	Violates the Single Responsibility Principle (SRP). The `SporeBeast` class would be responsible for its own attributes, generating actions, evaluating complex transition conditions for every state, and executing entry effects.
Avoids the overhead of creating multiple new classes and interfaces for states.	Extremely difficult to maintain and scale. Adding a new state or modifying a transition condition would require modifying the core `SporeBeast` class, violating the Open-Closed Principle (OCP). The `playTurn` method would become heavily bloated.

Design 2

Implement the State Design Pattern by creating a `CreatureState` interface and representing each state (e.g., `CamouflagedState`) as a concrete class that implements this interface.

Pros	Cons
Strongly adheres to the Single Responsibility Principle (SRP). Each concrete state class (like `CamouflagedState`) is solely responsible for its specific turn actions, deterministic transition logic, and unique entry effects.	Increases the number of classes in the project, which might slightly increase the initial complexity of the system architecture.
Adheres to the Open-Closed Principle (OCP). New states can be added simply by creating new classes that implement `CreatureState`, without modifying the `SporeBeast` class or existing state classes.	
Promotes code reusability and clean architecture. The `SporeBeast` class acts only as a context, delegating behavior to the `CreatureState` interface, reducing tight coupling.	

Conclusion

The new `CamouflagedState` created implements the `CreatureState` interface. Since the requirement demands that creatures can change between at least four distinctly implemented states with complex, deterministic transition logic, it is logical to abstract these behaviors into separate state classes. The SporeBeast class now solely uses a currentState pointer to delegate the playTurn logic. This ensures that `SporeBeast` can seamlessly switch its behavior at runtime by changing the state object it points to, without knowing the concrete implementation details of each state (Liskov Substitution Principle).

The main SporeBeast class has a dependency on the abstract CreatureState interface rather than concrete state classes like CamouflagedState. SporeBeast is a high-level module defining the creature's existence in the game, whilst the specific states are considered low-level modules implementing detailed behavioral logic (like hiding, scanning for threats, or

counting turns). Removing dependencies of SporeBeast on these concrete state classes reduces the risk of the core creature logic being affected by changes in individual state behaviors or the addition of new states (Dependency Inversion Principle). Furthermore, encapsulating the transition logic (getNextState) and entry effects (onEnter) within the state interface guarantees that the strict rules of REQ5 (at most one transition per turn, effect triggering on change) are managed systematically by the framework.